该文档实现了一种基于格的后量子累加器，核心算法为
Add,Delete,MemWitUp,MemVer,MemWitSync。

其他的算法，如 F_k(x,acc)， EvalF ,EvalFx,SampleLeft 都是核心算法需要的前置算法。

参数值：n=1， $\ell = 32$,k 是 128bit,d=2096,q 约为2^{100},m=10, m'约为 100。文档里面有部分参数值不太对， 但是这个文档和/home/wkk/pq_ac/lattice-anonymous-credentials 里的参数不完全相同，你可以自己调一下。

Given let $\tilde{q} = \lceil \log_2 q \rceil$ and

$$\mathbf{g}^\top = \left(1, \mathbf{2}, \mathbf{2}^2 \ldots, \mathbf{2}^{\tilde{q}-1}\right) \in R_q^{1\times\tilde{q}}$$

The gadget matrix of base b is defined by $\mathbf{G} = \mathrm{I}_n \otimes \mathbf{g}^\top \in R_q^{n\times m}$. In addition, there exists an efficient function $\mathbf{G^{-1}}: R_q^{n\times t} \to R_q^{m\times t}$ such that for any $\mathbf{A} \in R_q^{n\times t}$, we have $\mathbf{G}\cdot\mathbf{G^{-1}}(\mathbf{A}) = \mathbf{A}$.

$\boldsymbol{Algorithm}: \boldsymbol{F_k(x, acc)} \to \mathbf{r}:$

这是一个伪随机函数，对于同一个主密钥 k，只要输入的参数（如特定的 x 和 acc）完全一致，$F_k(x, acc)$ 无论执行多少次，计算多少次，输出的随机数种子 r 必定 100%相同。

$\boldsymbol{Algorithm}$: $\mathbf{EvalF}(\mathbf{1_x}, \mathbf{B})$ $\boldsymbol{and}$ $\mathbf{EvalFx}(\mathbf{1_x}, \mathbf{B}, \mathbf{y})$

/* 对于$\mathrm{EvalF}(1_{\mathbf{x}}(\mathbf{y}), \mathbf{B}) \to \mathbf{B}_{1_{\mathbf{x}}(\mathbf{y})}$, $\mathrm{EvalFx}(1_{\mathbf{x}}(\mathbf{y}), \mathbf{B}, \mathbf{y}) \to \mathbf{H}_{1_{\mathbf{x}}(\mathbf{y}),y}$, 目的是生成满足 $(B - x^T \otimes G)\cdot H_{1_{\mathbf{x}}(\mathbf{y}),y} = B_{1_{\mathbf{x}}(\mathbf{y})} - 1_{\mathbf{x}}(\mathbf{y})\cdot G$和 $\left\|H_{1_{\mathbf{x}}(\mathbf{y}),y}\right\|_\infty \le b-1$这两个条件的$\mathbf{B}_{1_{\mathbf{x}}(\mathbf{y})}$和$\mathbf{H}_{1_{\mathbf{x}}(\mathbf{y}),y}$ */

For any $\ell \in \mathbb{N}$, binary vectors $\mathbf{x}, \mathbf{y} \in \{0,1\}^\ell$, and matrix $\mathbf{B} = [\mathbf{B}_0 \mid \cdots \mid \mathbf{B}_{\ell-1}] \in R_q^{n\times m\ell}$, $\mathbf{x}$=$\{x_0, x_1 \cdots x_{\ell-1}\}$, $\mathbf{y}$=$\{y_0, y_1 \cdots y_{\ell-1}\}$, let

$$1_{\mathbf{x}}(\mathbf{y}) = \begin{cases} 1 & \text{if } \mathbf{x} = \mathbf{y} \\ 0 & \text{otherwise} \end{cases}, \quad \mathbf{D}_i = \begin{cases} \mathbf{B}_i & \text{if } x_i = 1 \\ \mathbf{G} - \mathbf{B}_i & \text{if } x_i = 0 \end{cases}, \text{ and } \delta_{x,y} = \begin{cases} 1 & \text{if } x = y \\ 0 & \text{otherwise} \end{cases}$$

We have $\mathbf{EvalF}(\mathbf{1_x}, \mathbf{B}) = \mathbf{D}_0 \cdot \mathbf{G}^{-1}\left(\mathbf{D}_1 \cdot \mathbf{G}^{-1}\left(\mathbf{D}_2 \cdot \mathbf{G}^{-1}\left(\cdots \mathbf{G}^{-1}(\mathbf{D}_{\ell-1})\right)\right)\right)$ and

$$\mathbf{EvalFx}(\mathbf{1_x}, \mathbf{B}, \mathbf{y}) = \begin{bmatrix} (-1)^{(1-x_0)} \cdot \mathbf{G}^{-1}\left(\mathbf{D}_1 \cdot \mathbf{G}^{-1}\left(\mathbf{D}_2 \cdots \mathbf{G}^{-1}(\mathbf{D}_{\ell-1})\right)\right) \\ (-1)^{(1-x_1)} \cdot \delta_{x_0,y_0} \cdot \mathbf{G}^{-1}\left(\mathbf{D}_2 \cdot \mathbf{G}^{-1}\left(\cdots \mathbf{G}^{-1}(\mathbf{D}_{\ell-1})\right)\right) \\ \vdots \\ (-1)^{(1-x_{\ell-2})} \cdot \delta_{x_0,y_0}\delta_{x_1,y_1} \cdots \delta_{x_{\ell-3},y_{\ell-3}} \cdot \mathbf{G}^{-1}(\mathbf{D}_{\ell-1}) \\ (-1)^{(1-x_{\ell-1})} \cdot \delta_{x_0,y_0}\delta_{x_1,y_1} \cdots \delta_{x_{\ell-3},y_{\ell-3}}\delta_{x_{\ell-2},y_{\ell-2}} \cdot \mathbf{I}_m \end{bmatrix}$$

$\boldsymbol{Algorithm}: \boldsymbol{SampleLeft}(\boldsymbol{A}, \boldsymbol{M_1}, \boldsymbol{R_A}, \boldsymbol{u}, \mathfrak{s}; \boldsymbol{r_a})$

1. 采样一个满足条件足够短的向量 e_2

2. 得到$e_1 \xleftarrow{R}\text{TSamplePre}(R_1, R_2, A_a, u - (M_1 \cdot e_2), t, \mathfrak{s}, r_a)$， $R_A = (R_1, R_2)$

3. output $e \leftarrow (e_1, e_2)$

$Output$: $F_1 := (A \mid M_1)$.

这个算法底层核心采样思想来自/home/wkk/truncated-sampler/signature_truncated 的 truncated-sampler 算法，但是实际在实现时需要进行更改算法或参数，根据 truncated-sampler 算法重新写一个 TSamplePre 算法，r_a代表来自算法外部输入的随机种子，当包括r_a

的所有输入相同时，输出相同，即算法是确定性算法。其中M_1在传入时应该是 $B_a - x^T \otimes G$，A传入时应该是$[I_d | A_a | TG_H - (R_1 + A_a R_2)]$。最终实现了 $e \in \Lambda_q^u(F_1)$，$(A \mid M_1) \cdot e = u \ mod\, q$。

以下算法的参数完全统一，包括输入输出的参数。

其中，*$\mathcal{A}$和acc为累加器值，s_x*为见证，*x是撤销句柄，upmsg是广播消息，*

A应该是$[I_d|A_a|TG_H-(R_1+A_aR_2)]$不确定你再查看一下，

$\boldsymbol{Algorithm:AccGen(pp)}$//参数与陷门生成

(1)$Compute\ \ (A_a,R_A)\leftarrow TrapGen(1^n,1^m,q)$ 实际上因为底层调用来自truncated − sampler 算法，所以A_a与R_A的生成应该查看truncated − sampler 算法的矩阵与陷门生成

(2)$Sample\ \ B_a\leftarrow R_q^{n\times \ell m'},(acc_0,u_a)\leftarrow R_q^{2n}$

(3)$k_a\leftarrow(0,1^\lambda)$

(4)$Initialize\ an\ empty\ dictionary\ DX:M\rightarrow\{Add,Del,\perp\}\times N\ and\ a\ counter\ ctr\coloneqq 0.$

(5)$Set\ \ \mathcal{A}_0\coloneqq acc_0,and\ \ st_a\coloneqq(k_a,ctr,acc_0,DX).$

$Output:(R_A,\mathcal{A}_0,st_a).$

$\boldsymbol{Algorithm:Add(A,B_a,R_A,st_a,\mathcal{A},x)}$//句柄 x 对应者加入累加器

(1)$(k_a,ctr,acc_0,DX)\coloneqq st_a,acc\coloneqq\mathcal{A},x\in M$

(2)$Parse\ \ (Op,i)\coloneqq DX[x],and\ \ if\ \ Op\neq\perp,abort.$ $x\ was\ already\ added\ or\ deleted$

(3)$Compute\ \ r_a\leftarrow F_{k_a}(x,acc).$

(4)$Compute\ \ s_x\leftarrow SampleLeft(A,B_a-x^\top\otimes G,R_A,acc,s;r_a).$

(5)$Set\ \ DX[x]\coloneqq(Add,ctr),,upmsg=(Add,\perp).$

(6)$Set\ \ st_a'\coloneqq(k_a,ctr,acc_0,DX)$

$Output:(\mathcal{A},s_x,upmsg,st_a').$

$\boldsymbol{Algorithm:Delete(A,B_a,u_a,R_A,st_a,\mathcal{A},x,s_x)}$

//累加器删除句柄 x 对应者，且更新累加器值

(1)$(k_a, ctr, acc_0, DX) \coloneqq st_a, acc \coloneqq \mathcal{A}, x \in M$

(2)$If\ \ ctr > \gamma, abort.$

$\square$ $We\ \ have\ \ exceeded\ \ the\ \ allowed\ \ number\ \ of\ \ deletions$

(3)$Parse\ \ (Op, i) \coloneqq DX[x], and\ \ if\ \ Op \neq Add, abort.$

$\square$ $x\ \ was\ \ already\ \ deleted\ \ or\ \ never\ \ added$

(4)$Compute\ \ B_{1_x} \leftarrow EvalF(1_x, B_a)$

(5)$acc' \leftarrow acc + B_{1_x} \cdot G^{-1}(u_a)$

(6)$Set\ \ ctr' \leftarrow ctr + 1\ \ and\ \ \mathcal{A}' \coloneqq acc'.$

(7)$Set\ \ DX[x] \coloneqq (Del, ctr), upmsg \coloneqq (Del, x), and\ \ st' \coloneqq (k_a, ctr', acc_0, DX).$

$Output: (\mathcal{A}', upmsg, st_a').$

$\boldsymbol{Algorithm: MemWitUp(A, B_a, u_a, x, s_x, upmsg)}$

//成员根据广播内容更新自身见证

(1)$(Op, y) \coloneqq upmsg$

(2)$If\ \ Op = Add: do\ \ nothing.$

$Else, if\ \ Op\ \ =\ \ Del:$

$-Compute\ \ H_{1_{y,x}} \leftarrow EvalFx(1_y, B_a, x)$

$-s_x' \leftarrow s_x + \left[0_{m' \times m} \mid H_{1_{y,x}}^{\top}\right]^{\top} \cdot G^{-1}(u_a)$

$-Output: s_x'$

$\boldsymbol{Algorithm: MemVer(A, B_a, u_a, \mathcal{A}, x, s_x)}$

//成员见证与句柄 x 的验证

(1)$acc \coloneqq \mathcal{A}, do\ \ the\ \ following:$

$If\ \ [A \mid B_a - x^{\top} \otimes G] \cdot s_x = acc\ \ and\ \ \|s_x\| \leq \beta, return\ \ 1.$

$Otherwise, return\ \ 0.$

$\boldsymbol{Algorithm: MemWitSync(A, B_a, u_a, R_A, st_a, \mathcal{A}, x)}$

//为成员重新创建一个，与当时发放的见证一模一样的见证

(1)$Given\ \ (k_a, ctr, acc_0, DX) \coloneqq st, acc \coloneqq \mathcal{A}$

(2)$Parse\ \ (Op, i) \coloneqq DX[x], and\ \ if\ \ Op \neq Add, abort.$ $x\ \ is\ \ not\ \ in\ \ the\ \ accumulator$

(3)$Let\ \ Z_0 = y \in \mathcal{M}: (Op, j) \coloneqq DX[y], Op = Del, 0 < j \leq i\ \ .$

$The\ \ set\ \ of\ \ elements\ \ deleted\ \ before\ \ x\ \ was\ \ added$

(4)$Let\ \ Z_1 = y \in \mathcal{M}: (Op, j) \coloneqq DX[y], Op = Del, i < j \leq ctr.$

$The\ \ set\ \ of\ \ elements\ \ deleted\ \ after\ \ x\ \ was\ \ added$

(5)$Compute\ \ acc_i \leftarrow acc_0 + \sum_{y \in Z_0} EvalF(1_y, B_a) \cdot G^{-1}(u_a).$

$Were\ \ cover\ \ the\ \ accumulator's\ \ value\ \ from\ \ when\ \ x\ \ was\ \ added$

(6)$Compute\ \ r_a \leftarrow F_{k_a}(x, acc_i).$

(7)$Compute\ \ s_x \leftarrow SampleLeft(A, B_a - x^{\top} \otimes G, R_A, acc, s; r_a).$

(8)$For\ \ each\ \ y \in Z_1, do:$

$-Compute\ \ H_{1_y,x} \leftarrow EvalFx(1_y, B_a, x).$

$-s_x \leftarrow s_x + \left[0_{m' \times m} \middle| H_{1_y,x}^{\top}\right]^{\top} \cdot G^{-1}(u_a).$

$Output: s_x$